

Packages in Java

A **package** in Java is a namespace that organizes classes and interfaces into manageable groups. It is a way to bundle related classes and interfaces together, providing a way to avoid name conflicts and better modularization of code.

Why Use Packages?

1. **Organization:** Packages group related classes and interfaces, making the codebase easier to understand and maintain.
 2. **Avoid Name Conflicts:** Packages allow classes with the same name to coexist in different packages without conflicts.
 3. **Access Control:** Packages help control access to classes, methods, and fields using access modifiers (`public` , `protected` , etc.).
 4. **Reusability:** Predefined classes in existing packages (e.g., `java.util`) can be reused in your program.
 5. **Encapsulation:** Packages allow encapsulation by providing access protection.
-

Types of Packages

1. **Built-in Packages:**
 - Java provides several predefined packages like:
 - `java.lang` (default package, no need to import explicitly)
 - `java.util` (contains utility classes like `ArrayList` , `HashMap` , etc.)
 - `java.io` (classes for input/output operations)
 - `java.net` (classes for networking)
 - `java.sql` (classes for database connectivity)
 2. **User-defined Packages:**
 - Created by developers to organize their own code.
-

How to Create a Package

To create a package, use the `package` keyword as the first statement in the Java file.

Steps to Create a Package:

1. Declare the package name at the beginning of the file.
 2. Save the file in a directory matching the package name.
 3. Compile the file with `javac -d` to specify the destination of the compiled files.
-

Example: Creating and Using a Package

Step 1: Create a Package

```
// Save this file as Example.java in a directory named "mypackage"
package mypackage;

public class Example {
    public void displayMessage() {
        System.out.println("Hello from the Example class in mypackage!");
    }
}
```

Step 2: Compile the Package

Run the following command to compile the file:

```
javac -d . Example.java
```

This creates a `mypackage` directory in the current folder containing the compiled `Example.class` file.

Step 3: Use the Package

Create another class in a different file to use this package.

```
// Save this file as Main.java
import mypackage.Example;

public class Main {
    public static void main(String[] args) {
        Example obj = new Example(); // Create an object of Example class
        obj.displayMessage();         // Call the method
    }
}
```

Step 4: Compile and Run

Compile the `Main.java` file:

```
javac Main.java
```

Run the program:

```
java Main
```

Output:

```
Hello from the Example class in mypackage!
```

Using Built-in Packages

To use a built-in package, simply import it using the `import` keyword.

Example: Using the `java.util` Package

```
import java.util.ArrayList;

public class BuiltInPackageDemo {
    public static void main(String[] args) {
        // Create an ArrayList
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");

        // Print the ArrayList
        System.out.println("Fruits: " + list);
    }
}
```

Output:

```
Fruits: [Apple, Banana, Cherry]
```

Access Control in Packages

Access modifiers control the visibility of classes, methods, and fields across packages:

1. `public` : Accessible from any package.
 2. `protected` : Accessible within the same package and subclasses in other packages.
 3. **Default (no modifier)**: Accessible only within the same package.
 4. `private` : Accessible only within the class.
-

Package Naming Conventions

1. **Reverse Domain Names**: Use your domain name in reverse to avoid conflicts with other packages (e.g., `com.example.mypackage`).
 2. **Lowercase Letters**: Package names should be in lowercase to avoid conflicts with class names.
-

Key Points About Packages

1. **Default Package**: If no package is declared, the class belongs to the default package. Classes in the default package cannot be imported.
2. **Wildcard Imports**: Use `import package_name.*;` to import all classes in a package.
 - Example: `import java.util.*;`
3. **Static Imports**: Import static members (methods or variables) of a class.
 - Example:

```
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;

public class StaticImportDemo {
    public static void main(String[] args) {
        System.out.println("Value of PI: " + PI);
        System.out.println("Square root of 16: " + sqrt(16));
    }
}
```

Summary Table

Aspect	Description
Definition	A namespace that organizes classes and interfaces into manageable groups.

Aspect	Description
Types	Built-in packages (<code>java.util</code> , <code>java.io</code> , etc.) and user-defined packages.
Keyword	<code>package</code> (used to define a package at the beginning of a file).
Compilation	Use <code>javac -d</code> to compile files with packages.
Usage	Use <code>import package_name.ClassName;</code> or <code>import package_name.*;</code> .
Access Control	Controlled using modifiers like <code>public</code> , <code>protected</code> , <code>default</code> , and <code>private</code> .
Naming Convention	Use reverse domain names and lowercase letters.